`ROWS BETWEEN` defines which rows are included in the window frame for a window function calculation.

In simple terms, it tells SQL:

> For the current row, which rows should I consider when performing the calculation?

It is used inside the `OVER()` clause.

1. Basic syntax

```
FUNCTION_NAME(column) OVER (
    PARTITION BY column
    ORDER BY column
    ROWS BETWEEN start_point AND end_point
)
```

The frame is defined relative to the current row.

2. Understand the different boundaries

Boundary	Meaning
`UNBOUNDED PRECEDING`	Start from the first row in the partition
`n PRECEDING`	Go back n rows from the current row
`CURRENT ROW`	The current row
`n FOLLOWING`	Go forward n rows from the current row

`UNBOUNDED FOLLOWING`	Go until the last row in the partition

3. Example: Running total

Consider this sales data:

Month	Sales
Jan	100
Feb	200
Mar	300
Apr	400
May	500

Query:

```
SELECT
  month,
  sales,
  SUM(sales) OVER (
    ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING
```

```
        AND CURRENT ROW
    ) AS running_total
FROM monthly_sales;
```

Result:

Month	Sales	Running total
Jan	100	100
Feb	200	300
Mar	300	600
Apr	400	1000
May	500	1500

How does it work?

For the March row

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Jan

100

Feb

200

Mar

300

100 + 200 + 300 = 600

Only rows from the beginning up to the current row are included.

4. Example: Moving average of the last 3 rows

```
SELECT
    month,
    sales,
    AVG(sales) OVER (
        ORDER BY month
        ROWS BETWEEN 2 PRECEDING
                AND CURRENT ROW
    ) AS moving_avg
FROM monthly_sales;
```

Result:

Month	Sales	Rows considered	Moving average
Jan	100	Jan	100
Feb	200	Jan, Feb	150
Mar	300	Jan, Feb, Mar	200

Apr	400	Feb, Mar, Apr	300
May	500	Mar, Apr, May	400

Notice that the first two rows have fewer than three rows available.

5. More useful frame combinations

Window frame	Meaning
`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`	Running calculation
`ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`	Current row + previous 2 rows
`ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING`	Previous + current + next row
`ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING`	Current row through last row
`ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`	Entire partition

6. Important: ROWS vs RANGE

SQL also supports `RANGE BETWEEN`.

- `ROWS` works with physical row positions.

- `RANGE` works with logical ordering values and includes rows with matching order values within the specified range.

For example, consider:

Employee	Salary
Amit	50000
Rahul	60000
Priya	60000
Neha	70000

```
SUM(salary) OVER (
   ORDER BY salary
   ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

With `ROWS`, each physical row is processed sequentially.

```
SUM(salary) OVER (
   ORDER BY salary
   RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)
```

With `RANGE`, employees with the same salary are peers and receive the same cumulative result.

Employee	Salary	ROWS total*	RANGE total
Amit	50000	50000	50000
Rahul	60000	110000	170000
Priya	60000	170000	170000
Neha	70000	240000	240000

*The ROWS result shown assumes Rahul appears before Priya. Without a tie-breaker, their order is not guaranteed.

7. A simple way to remember

- `PARTITION BY` → Which group?
- `ORDER BY` → In what order?
- `ROWS BETWEEN` → Which rows should participate in the calculation?
- `SUM / AVG / FIRST_VALUE / LAST_VALUE` → What calculation or value should be returned?

One important interview point: `ROWS BETWEEN` is particularly useful for running totals, moving averages, rolling sums, and controlling the behavior of `LAST_VALUE()`.